API Gateway & Load Balancer - Integration Patterns in Microservices

When we build a system with multiple microservices, clients want to communicate with our system.

For example:

Imagine we are building a university management system with multiple microservices:

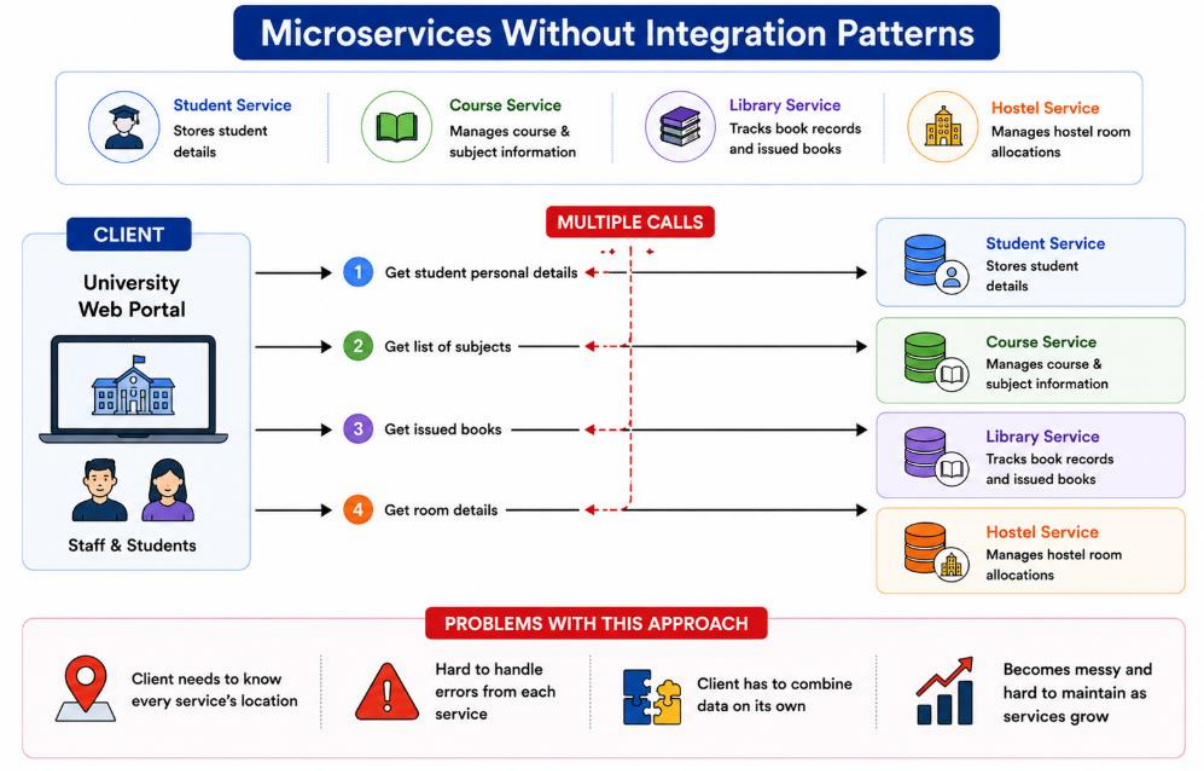
1. Student Service: stores student details
2. Course Service: manages course and subject information
3. Library Service: tracks book records and issued books
4. Hostel Service: manages hostel room allocations

The client here is the university web portal used by staff and students.

If the client talks to each service directly:

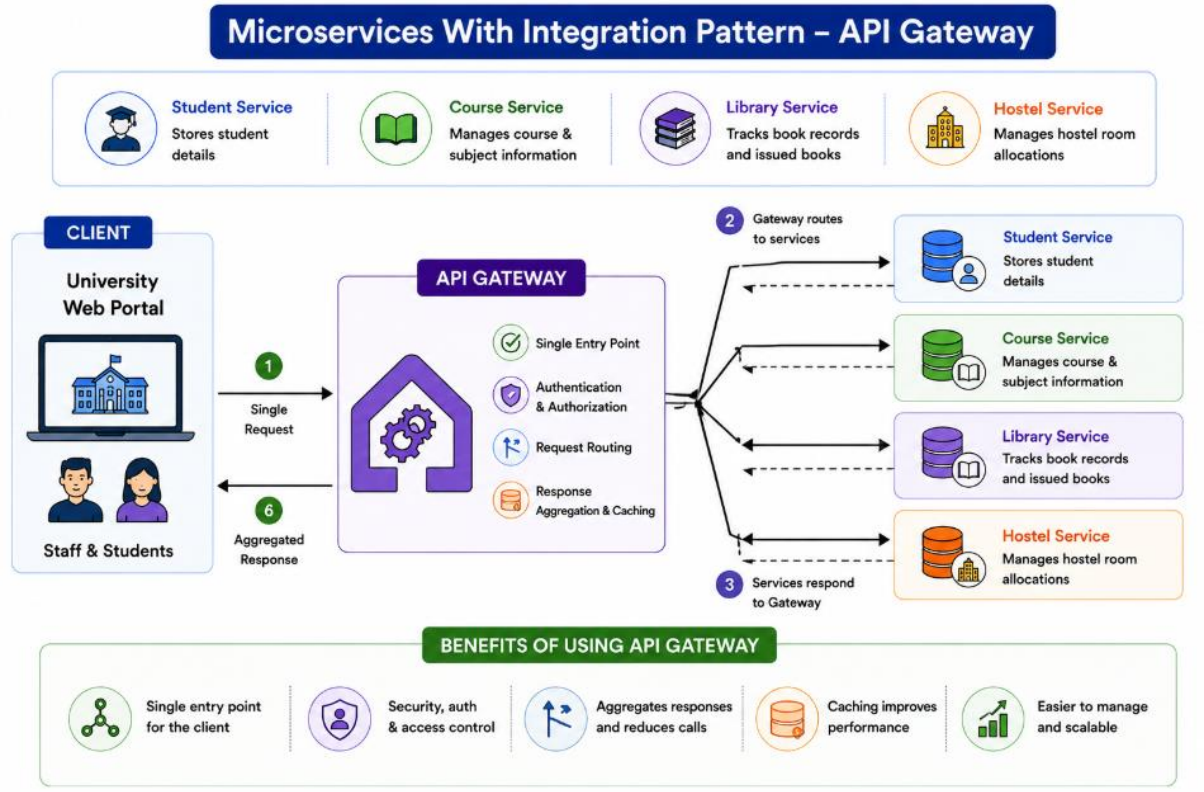
1. The portal calls Student Service to get student personal details.
2. The portal calls Course Service to get the list of subjects.
3. The portal calls Library Service to get which books are issued.
4. The portal calls Hostel Service to get room details.

Every time someone logs in, the portal has to make four different calls to collect all the data. This setup works but becomes messy when the number of services grows.



The client needs to know every service's location, handle errors from each, and combine data on its own.

Note: Big companies like **Netflix** and **Uber** also started this way when their systems were small, then later moved to more centralized ways to reduce client complexity. This is exactly why many teams introduce API Gateway or Aggregator patterns later.



What is API gateway pattern and why we use it?

When we build a system with many microservices, it's hard for the client to talk to each one. The client needs to know all service locations, handle multiple calls, and combine the data. This is where the **API Gateway** pattern helps.

An API Gateway works like the single main entrance to a building.Instead of letting everyone enter through different doors, we create one main door. Inside the building, people can go to different rooms. Similarly, the gateway takes the client's request and sends it to the right microservice.

Real world example: Online Shopping Portal

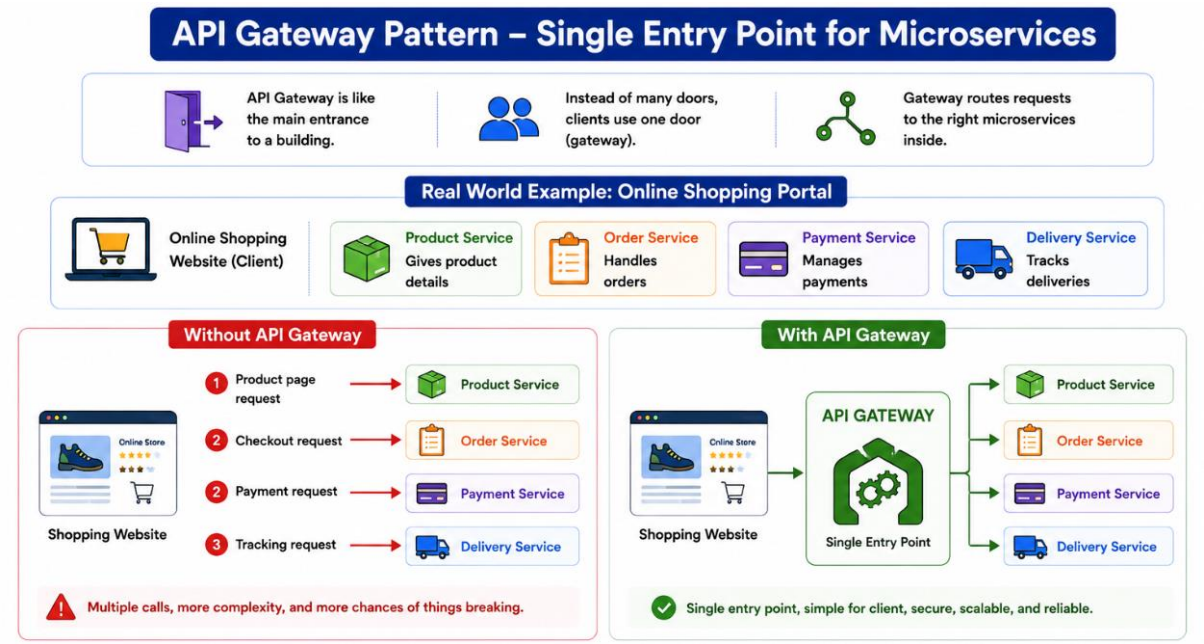
Let's say we build an online shopping portal with these microservices:

1. Product Service – gives product details.
2. Order Service – handles orders.
3. Payment Service – manages payments.
4. Delivery Service – tracks deliveries.

Without an API Gateway, the shopping website would need to talk to each of these services directly.

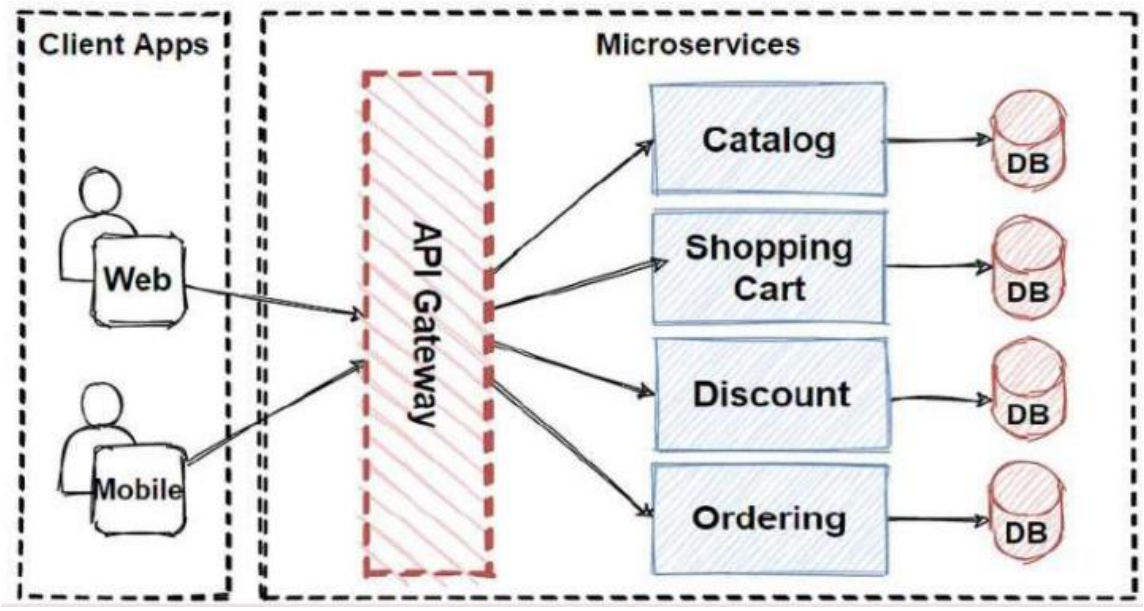
- Product page calls Product Service.
- Checkout page calls Order Service and Payment Service.
- Delivery tracking page calls Delivery Service.

That means multiple calls, more complexity, and more chances of things breaking.



With an API Gateway, the website talks to just one gateway.

5. The gateway receives the product page request and sends it to Product Service.
6. It receives checkout requests and passes them to Order and Payment Services
7. It receives tracking requests and passes them to Delivery Service.



What are the features of API gateway?

An API Gateway is more than just a single-entry point for microservices. It provides several features that make a system easier to manage, faster, and more secure.

Let's explain these features with an online shopping portal example.

1. Request Routing : The gateway decides which microservice should handle a client request. Example:

- When a user opens the product page, the gateway sends the request to Product Service.
- When the user checks out, the request goes to Order Service and Payment Service. The client doesn't need to know where the services are or how to call them.

2. Load Balancing : The gateway can distribute requests evenly among multiple instances of a microservice.

Example:

- If thousands of users open the product page at the same time, the gateway sends requests to different Product Service instances so that no single instance is overloaded.

3. Authentication and Security : The gateway can check if a request is valid and secure before letting it reach the services.

Example:

- Only logged-in users can place orders.
- The gateway verifies the user's token before forwarding the request to Order Service.

4. Response Aggregation : Sometimes the client needs data from multiple services at once. The gateway can combine responses into one.

Example:

- On the checkout page, the client wants product details, delivery options, and user balance.
- Instead of calling three services separately, the gateway collects all the data and sends one response.

5. Protocol Translation : The gateway can translate requests and responses between different formats.

Example:

- The client may call the gateway using HTTP, but internally, the gateway can talk to a service using gRPC.

6. Caching : The gateway can store frequently requested data to reduce repeated calls to services.

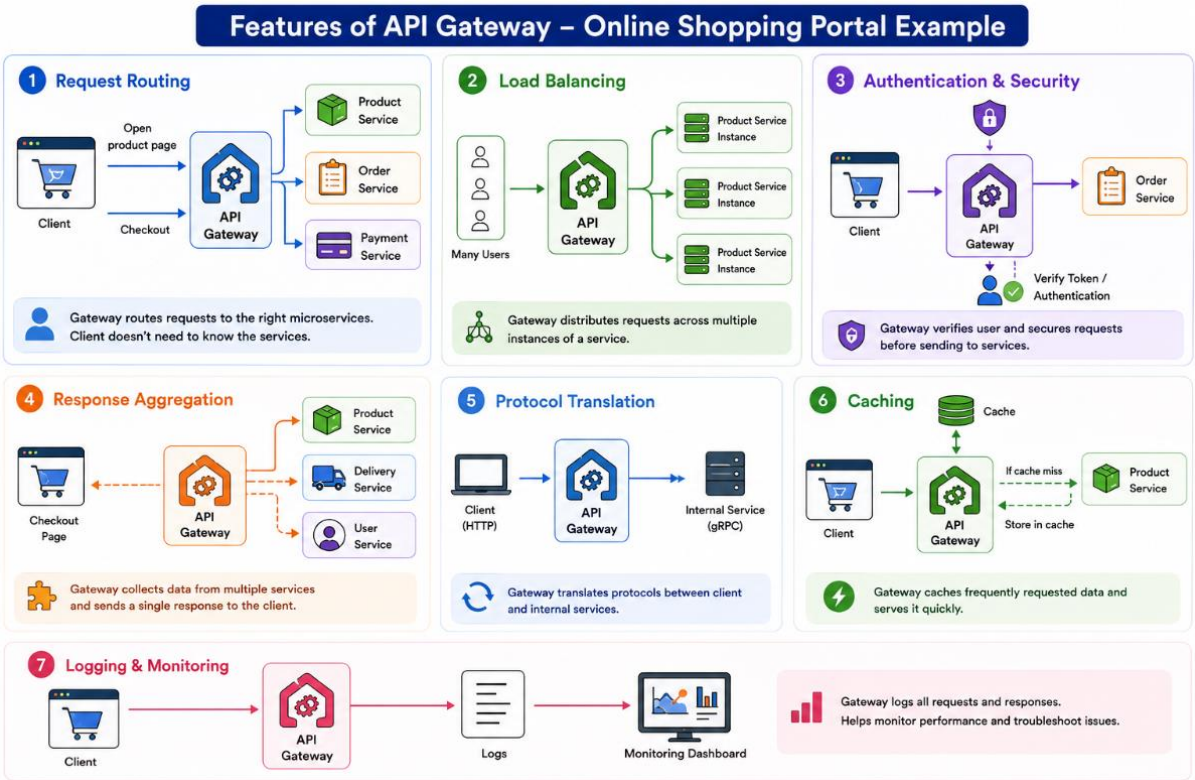
Example:

Product catalog data doesn't change often. The gateway can cache it and serve many users quickly without hitting the Product Service each time

7. Logging and Monitoring : The gateway can track all requests and help monitor system performance.

Example:

- If checkout requests fail often, the gateway logs it.
- Developers can use these logs to quickly find and fix issues.



What are the ways to implement API gateway in Spring?

In a system with many microservices, an API Gateway acts as the single-entry point for the client.

In Spring, we have simple ways to implement it. Let's see how, using an online shopping portal example.

1. Spring Cloud Gateway This is the official Spring solution to create an API Gateway. It works with Spring Boot applications and is easy to configure.
Example:

• The online shopping portal has Product Service, Order Service, Payment Service, and Delivery Service.

• We set up Spring Cloud Gateway to route requests:

1. /products/** → Product Service
2. /orders/** → Order Service
3. /payments/** → Payment Service
4. /delivery/** → Delivery Service

Features we get:

- Routing requests to the correct service
- Load balancing using Spring Cloud Load Balancer
- Security and authentication filters
- Logging and monitoring

This makes the client only talk to one URL instead of four separate services.

2. Zuul Gateway

Zuul is another popular API Gateway from Netflix, now integrated with Spring Cloud. It works similarly to Spring Cloud Gateway but was more widely used before Spring Cloud Gateway became standard.

Example:

- The shopping portal client calls /api/** on the Zuul Gateway.
- Zuul forwards requests to the correct microservice and can handle authentication, rate limiting, and logging.

3. Custom Spring Boot Gateway We can also create a custom gateway using Spring Boot by writing a small service that acts as a router

Example:

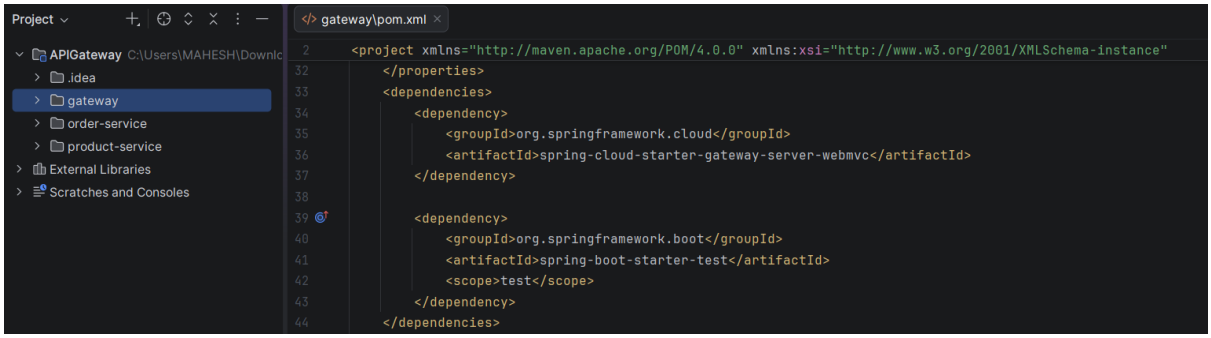
- The gateway service receives all requests at /api/**.
- It checks the path and forwards the request to the correct service using RestTemplate or WebClient.
- We can add security, caching, or response aggregation manually. This approach gives full control, but it requires more effort to implement features like load balancing or monitoring.

Ex: Real time Project Usecase

Let's say project contains 3 services.

1. Product service
2. Order service
3. API gateway: we have used spring-cloud-starter-gateway-server-webmvc dependency for gateway.

spring-cloud-starter-gateway-server-webmvc is a Spring Boot starter that enables the use of Spring Cloud Gateway with Spring WebMvc, providing a servlet-based implementation of the API Gateway.



we can also use **spring-cloud-starter-gateway-server-webflux** for reactive programming model.

1. Start Product Service which runs on port 8081.
2. Start Order Service which runs on port 8082.
3. Start API Gateway which runs on port 8080.

Client calls through the gateway:

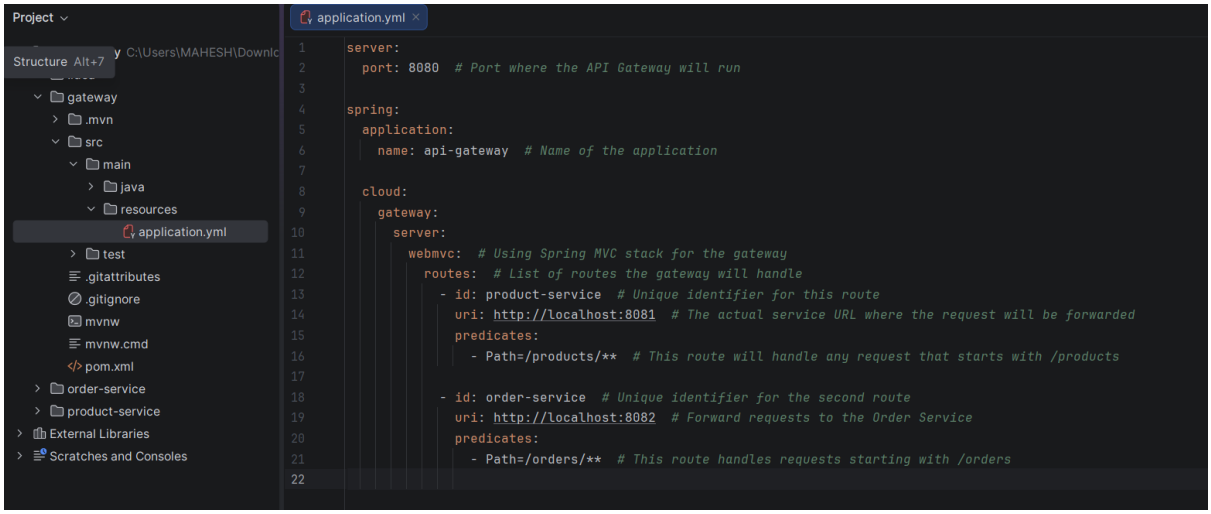
- GET http://localhost:8080/products → returns the list of products from Product Service
- POST http://localhost:8080/orders with body "Laptop" → returns "Order placed for: Laptop" from Order Service

How it works (You can test it from Postman):

Step 1: Client Sends Request The client calls the gateway: http://localhost:8080/products The client does not know about the internal services. It only talks to the Gateway.

Step 2: Gateway Receives Request

• Gateway is configured with routes in **application.yml**:




```yaml
server:
  port: 8080  # Port where the API Gateway will run

spring:
  application:
    name: api-gateway  # Name of the application


  cloud:
    gateway:
      server:
        webmvc:  # Using Spring MVC stack for the gateway
          routes:  # List of routes the gateway will handle
            - id: product-service  # Unique identifier for this route
              uri: http://localhost:8081  # The actual service URL where the request
will be forwarded
              predicates:
                - Path=/products/**  # This route will handle any request that starts
with /products

            - id: order-service  # Unique identifier for the second route
              uri: http://localhost:8082  # Forward requests to the Order Service
              predicates:
                - Path=/orders/**  # This route handles requests starting with /orders
```

Gateway sees the path /products/** and it should forward the request to Product Service.

Step 3: Gateway Forwards Request to Service

- The Gateway makes an internal HTTP call to the Product Service at http://localhost:8081/products.
- If the path was /orders/**, it would forward to http://localhost:8082/orders.

Step 4: Service Processes Request

1. Product Service receives the request.
2. It executes the controller method:
3. Service prepares the response (a list of products).

```java
@RestController
public class ProductController {

    @GetMapping("/products")
    public List<String> getProducts() {
        return List.of("Laptop", "Phone", "Tablet");
    }
}
```

Step 5: Gateway Receives Response

1. Gateway gets the response from Product Service.
2. Any filters configured at the Gateway (like adding headers, rewriting paths, logging) are applied here.

Step 6: Gateway Sends Response to Client

1. Gateway sends the final response back to the client.
2. The client sees the response as if it came directly from the gateway, unaware of the internal microservices.

Example Response: ["Laptop", "Phone", "Tablet"]

How API gateway works as load balancer?

In a microservice system, we often run multiple instances of the same service to handle more traffic and avoid downtime.

But then a question comes up: When a request comes in, how does it get distributed across these service instances? This is where the API Gateway acts like a load balancer.

It does not just forward requests but also distributes them smartly to keep the system fast and stable.

Let us say we have three instances of product-service running in three instances:

Instance	Address
product-service-1	localhost:8081
product-service-2	localhost:8082
product-service-3	localhost:8083

When a client sends a request to **/products**, we do not want all traffic to go to just one instance. Instead, we want to share the load. That is exactly what the API Gateway does automatically.

How It Works Internally

1. API Gateway receives a request.
2. It checks which service to forward the request to.
3. It gets a list of service instances from Eureka (or static config).
4. It applies load balancing strategy.
5. It selects one instance and sends the request to it. By default, it uses Round Robin, one request goes to instance 1, next to instance 2, and so on.

Tools for Load Balancing with API Gateway

Tool	Type	Purpose
Spring Cloud LoadBalancer	Client-side	Built-in load balancer in Spring
Eureka + LoadBalancer	Client-side	Service discovery + load balancing
API Gateway (Spring Cloud Gateway)	Server-side	Routes and load balances
Nginx	Server-side	Works as reverse proxy and load balancer
Kubernetes LoadBalancer	Cluster level	Manages load balancing in containers
Ribbon (Deprecated)	Client-side	Old Netflix load balancer

Let us see how load balancing works in a simple microservice setup using Spring Cloud Gateway and Eureka.

Step 1: pom.xml dependencies

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

Step 2: application.yml configuration

```yaml
spring:
  application:
    name: api-gateway

  cloud:
    gateway:
      routes:
        - id: product-service
          uri: lb://product-service //
          predicates:
            - Path=/products/**
```

Key Point: lb://product-service → here lb stands for load balancer.

This tells Spring Cloud Gateway:

"Send /products requests to product-service using load balancing."

So, if three instances of product-service are running, the Gateway will forward traffic to all of them in rotation using Spring Cloud Load Balancer.

What is difference between API gateway and load-balancer?

In a microservice system, both **API Gateway** and **Load Balancer** are used to handle incoming traffic, but they solve different problems.

Load balancer only distributes traffic across multiple servers or instances to improve performance and availability.

It does not understand business logic or API routes. It simply takes incoming requests and shares them across instances to avoid overload.

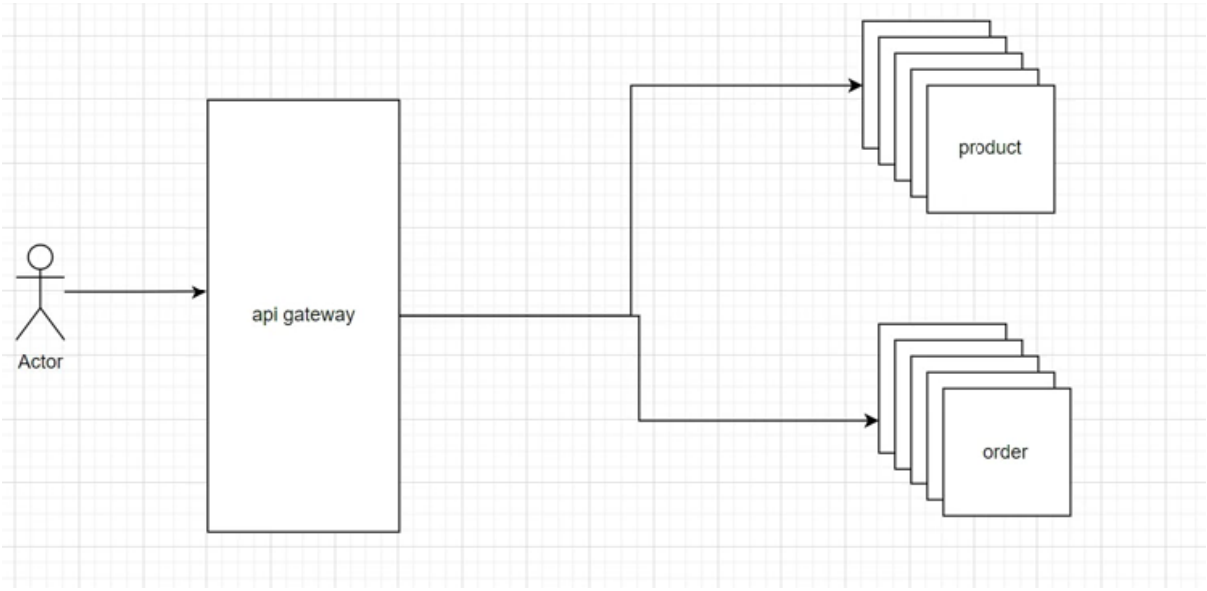
API Gateway is more powerful. It sits at the entry point of the system and controls how requests move to different microservices.

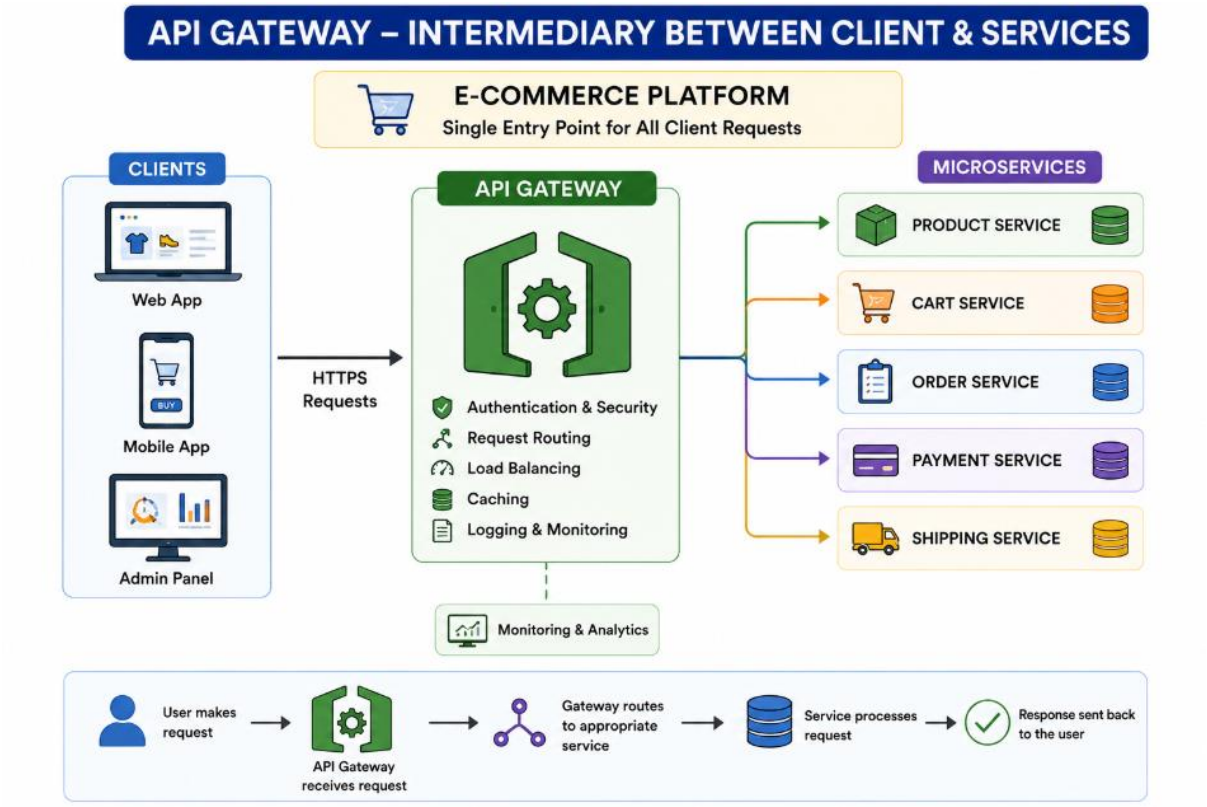
It can route requests to the correct service based on the URL path. It can also handle security, logging, authentication, request filtering, response transformation, and API composition.

Load balancer only works at the network level, but API Gateway works at the application level.

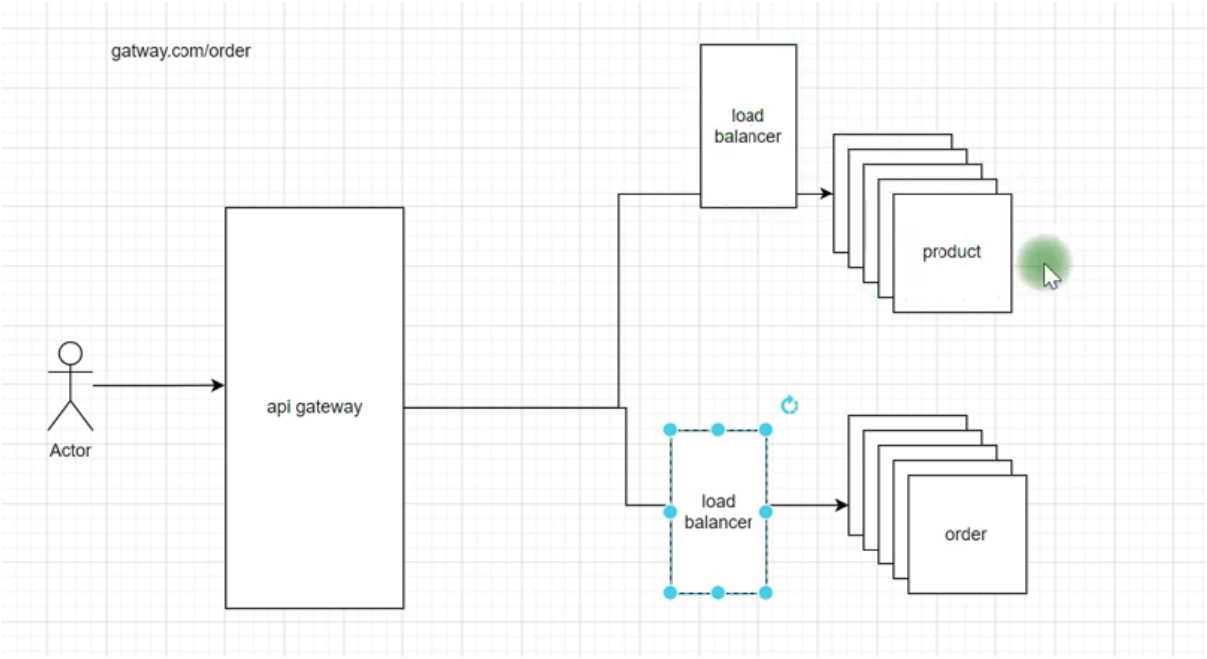
- **API gateway**
  - Acts as an intermediary between clients (such as mobile apps or web browsers) and a collection of backend services.
  - handles all of the traffic between the clients and the backend services
  - Provides a single point of entry for all API requests.
  - Also offers additional features such as rate limiting, authentication, and caching.

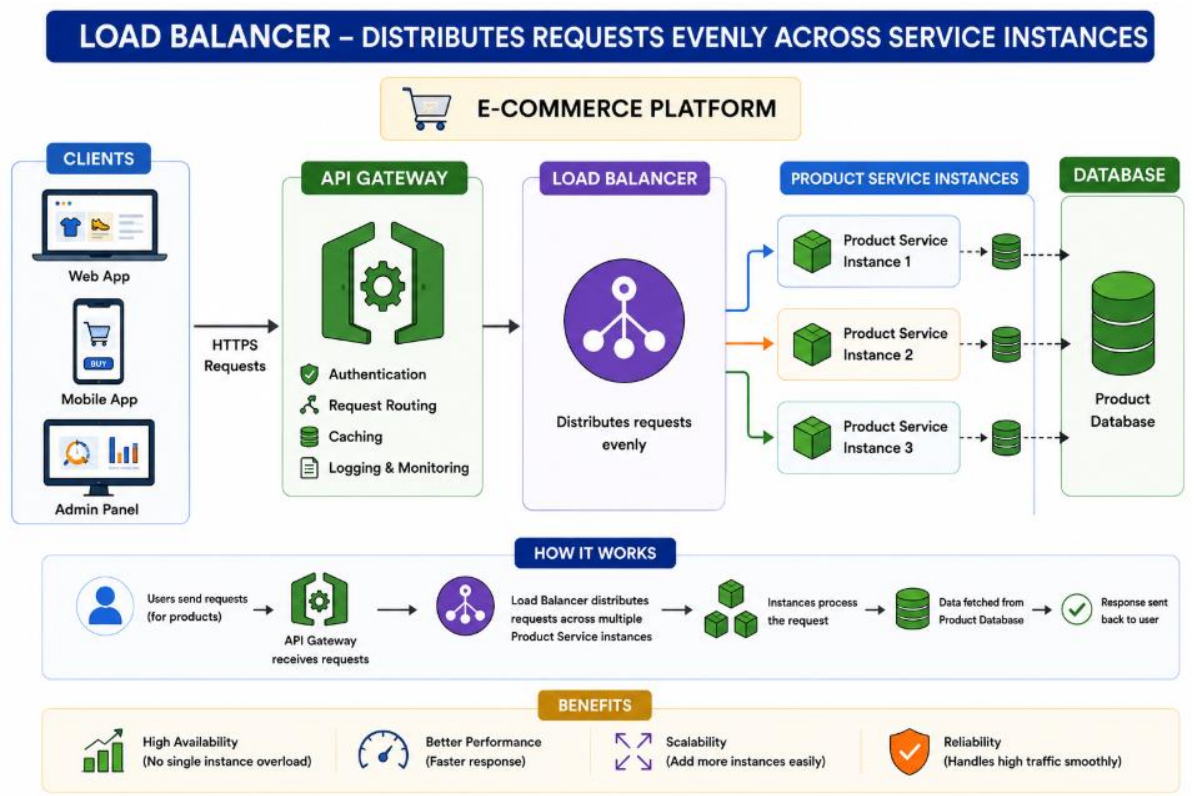
- **Load balancer**
  - distributes incoming network traffic across multiple servers, which helps prevent any one server from becoming overwhelmed with requests.





Load Balancer





Can API Gateway replace load balancer completely?

No, API Gateway cannot fully replace a load balancer. A load balancer manages traffic distribution, fault tolerance, and high availability at the network level, while an API Gateway handles routing, authentication, rate limiting, and cross-cutting concerns. They solve different problems and are often used together in production.

Real time uses of API Gateway?

1. Instagram - Managing Different Services Behind One Door

When we open Instagram, so many things happen at once. Our profile loads, new posts appear, stories show up, and notifications appear instantly. Each of these comes from a different microservice:

- Feed Service (for posts)
- Story Service (for stories)
- Notification Service (for likes and comments)
- Chat Service (for messages)
- User Service (for profile data)

Instead of the mobile app calling each of these services separately, all requests go through one single API Gateway. The Gateway receives the request, checks security, and routes it to the right service. It then combines the responses and sends them back to the app. This makes the app faster, lighter, and easier to maintain.

Questions about Load balancer

- What is a load balancer, and what purpose does it serve in distributed systems?
- What are some common algorithms used for load balancing, and how do they work?
- What are some considerations when choosing a load balancer for a distributed system?
- How do load balancers ensure high availability and fault tolerance?
- Can you explain the difference between a layer 4 and a layer 7 load balancer?
- How do you monitor the performance and health of a load balancer?
- How do you scale a load balancer for high traffic or increased demand?
- Can you give an example of a scenario where you would use a load balancer?

1. What is a Load Balancer and what purpose does it serve in distributed systems? Or

What is load balancer and how it plays important role in microservices?

In a microservice system, we never rely on just one instance of a service. If only one instance is running and it receives too many requests, it may slow down or even crash. To avoid this, we run multiple instances of the same service.

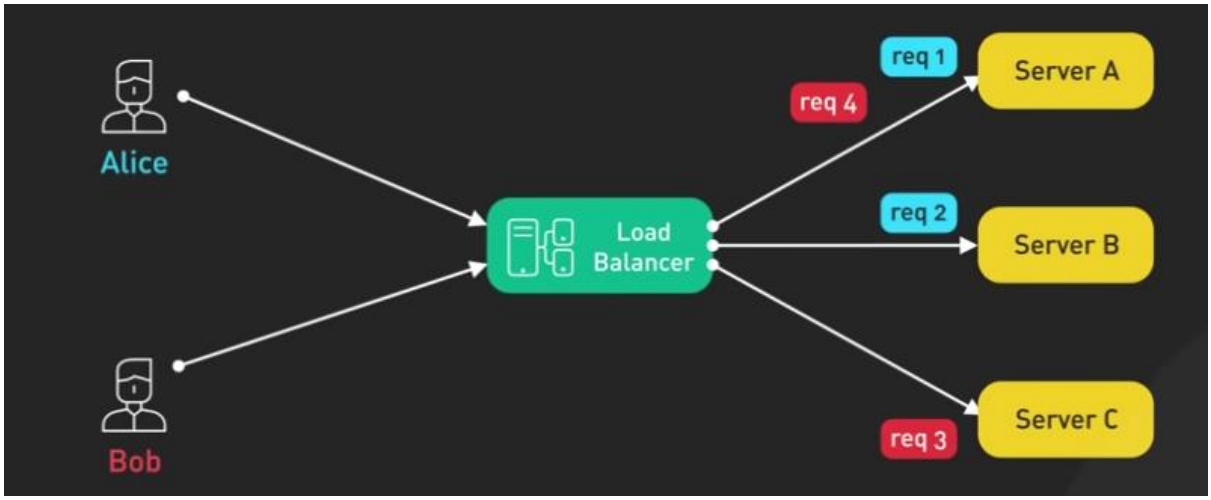
Example:

```
order-service running at port 8081
order-service running at port 8082
order-service running at port 8083
```

Now the challenge is: When a request comes, which instance should handle it? If all requests go to only one instance, that instance gets overloaded while others sit idle. This is where a Load Balancer becomes important.

What Is a Load Balancer

A load balancer is a component that distributes incoming requests across multiple service instances. It makes sure no single instance gets overloaded and traffic is shared fairly. Load balancer sits between the client and the microservices and plays the role of a traffic manager.



Load Balancer Tools Used in Spring Microservices

Tool	Type	Purpose
Spring Cloud LoadBalancer	Client-side	Built-in load balancer in Spring
Eureka + LoadBalancer	Client-side	Service discovery + load balancing
API Gateway (Spring Cloud Gateway)	Server-side	Routes and load balances
Nginx	Server-side	Works as reverse proxy and load balancer
Kubernetes LoadBalancer	Cluster level	Manages load balancing in containers
Ribbon (Deprecated)	Client-side	Old Netflix load balancer

2. What are common Load Balancing algorithms?

1. Round Robin

Requests are distributed sequentially.

Req1 → Server1

Req2 → Server2

Req3 → Server3

Req4 → Server1

2. Random Selection

As the name says, requests are sent to any instance randomly.

Example:

Requests may go like: `8082 > 8081 > 8083 > 8082 > 8081`

3. Least Connections

In this method, the request is assigned to the instance that has **the fewest active connections**. It is useful when each request takes a different amount of time to complete.

```
Current active requests: 8081 > 5 requests active
8082 > 2 requests active
8083 > 1 request active
Next request goes to -> 8083
```

4. Weighted Round Robin

This is a smarter version of Round Robin. If one service instance has **more power (CPU/RAM)**, we give it a **higher weight** so it can handle more traffic.

Example:

Weights:

```
8081 > weight 3
8082 > weight 1
8083 > weight 1
```

Order: 8081 > 8081 > 8081 > 8082 > 8083 > repeat: 8081 handles more traffic because it has higher capacity.

5. IP Hash In this method, the request always goes to an instance based on client IP address. So, the same user always connects to the same backend instance.

4. How do Load Balancers ensure High Availability and Fault Tolerance?

Health Checks

Load balancer continuously checks server health.

- Healthy → Receive Traffic
- Unhealthy → Removed

---

6. How do you monitor the health of a Load Balancer?

Metrics

- Request Count
- Response Time
- Throughput
- Active Connections
- Error Rate
- CPU Usage
- Memory Usage

Monitoring Tools

- Prometheus
- Grafana
- Datadog
- New Relic
- AWS CloudWatch

Health Checks

GET /health

Expected:

```
{
  "status":"UP"
}
```

---

7. How do you scale a Load Balancer?

Horizontal Scaling

Add more backend servers.

Auto Scaling

Multiple Load Balancers

---

8. Real-world scenario where Load Balancer is used?

Amazon Flipkart Big Billion Days

Millions of users open:

- Product pages
- Search pages
- Checkout pages

Without Load Balancer:

1. Users → One Server
2. Server crashes.

With Load Balancer:

- Traffic is distributed and application remains available.

Questions about API Gateway

- What is an API gateway, and what purpose does it serve in distributed systems?
- What are some common features offered by API gateways, and why are they important?
- How do API gateways handle traffic routing and service discovery in distributed systems?
- What are some common security concerns addressed by API gateways?
- How can you optimize API performance using an API gateway?
- Can you explain the difference between an API gateway and a service mesh?
- How do you monitor the performance and health of an API gateway?
- Can you give an example of a scenario where you would use an API gateway?
- How would you handle API authentication and authorization using an API gateway?
- How would you implement caching using an API gateway?
- How would you manage API versioning using an API gateway?

1. What is an API Gateway and what purpose does it serve?

API Gateway is a single-entry point for all client requests in a microservices architecture.

Instead of clients calling multiple services directly:

Gateway routes requests to appropriate services.

2. What are common features offered by API Gateways?

1. **Request Routing :** Routes request to correct service.
2. **Authentication :** JWT validation.
3. **Authorization :** Role-based access.
4. **Load Balancing :** Distributes traffic.
5. **Rate Limiting :** Prevents abuse.
6. **Caching :** Improves performance.
7. **Monitoring :** Tracks requests and errors.
8. **Response Aggregation :** Combines data from multiple services.

3. How does API Gateway handle Routing and Service Discovery?

Routing

- /products → Product Service

- /orders → Order Service
- /payments → Payment Service

Service Discovery

Gateway doesn't use fixed IPs.

Uses:

1. Eureka
2. Consul
3. Kubernetes Service Discovery

Example:

Gateway → Product Service

Gateway automatically discovers healthy instances.

4. What security concerns are addressed by API Gateway?

1. **Authentication :** Verify JWT tokens.
2. **Authorization :** Check permissions.
3. **SSL Termination :** HTTPS handling.
4. **Rate Limiting :** Prevent DDoS attacks.
5. **IP Filtering :** Block unwanted traffic.
6. **API Key Validation :** Validate API consumers.

5. How can API Gateway improve performance?

1. **Caching :** Store frequent responses.
2. **Load Balancing :** Distribute requests.
3. **Response Aggregation :** One call instead of multiple calls.
4. **Connection Reuse :** Reduce network overhead.
5. **Compression :** Smaller payloads.

6. Difference between API Gateway and Service Mesh?

Feature	API Gateway	Service Mesh
Handles	External Traffic	Internal Service Traffic
Entry Point	Yes	No
Authentication	Yes	Limited
Service-to-Service Security	Basic	Strong
Observability	Basic	Advanced

API Gateway: User → Gateway → Services

Service Mesh: Service A ↔ Service B ↔ Service C

7. How do you monitor API Gateway health?

Metrics

1. Request Count
2. Latency
3. Error Rate
4. Throughput
5. CPU Usage
6. Memory Usage

Tools

1. Prometheus
2. Grafana
3. CloudWatch
4. Datadog

Logs

1. Request Received
2. Request Routed
3. Response Sent

8. Real-world scenario for API Gateway?

E-commerce Application

Services:

1. Product Service
2. Order Service
3. Payment Service
4. Delivery Service

Without Gateway:

1. Client → Product Service
2. Client → Order Service
3. Client → Payment Service
4. Client → Delivery Service

Many connections.

With Gateway:

- Single entry point.
- Client doesn't need to know service locations.

9. How would you implement Authentication and Authorization using API Gateway?

Step 1: User Login

1. User authenticates.
2. Auth Service generates JWT.
3. JWT Token

Step 2: Client Calls Gateway

- Authorization: Bearer JWT

Step 3: Gateway Validates Token

Checks:

1. Signature
2. Expiry
3. Claims

Step 4: Authorization

1. ADMIN → /admin/*
2. USER → /products/*

Step 5: Forward Request

Only valid requests reach services.

Popular API Gateway Tools

- Spring Cloud Gateway
- Kong
- NGINX
- AWS API Gateway
- Azure API Management
- Apigee

Scenario Based Questions

1. You have a microservices-based application with several services that need to be exposed through a public API. How would you design the API Gateway and Load Balancing architecture to ensure scalability and high availability?

Answer :

I would place an API Gateway as the single entry point and put a Load Balancer in front of it.

Scalability

Add more API Gateway instances without changing clients.

Follow Up Question : Why LB goes in front of the API Gateway ?

The API Gateway is an application (like Spring Cloud Gateway, Kong, or Nginx) running on a server. Just like any other application, a single instance of an API Gateway has limits on how much CPU, memory, and network traffic it can handle.

If you have millions of users hitting **one** API Gateway instance directly, that gateway becomes a **Single Point of Failure (SPOF)** and a massive bottleneck. If it crashes, your entire ecosystem goes down.

To fix this, you run **multiple instances** of your API Gateway. To distribute user traffic evenly across those multiple gateway instances, you must place a high-capacity **Infrastructure Load Balancer** (like AWS ALB, F5, or Nginx at Layer 4) in front of them.

How It Works: The Step-by-Step Traffic Flow

Here is exactly how a request travels through this architecture in a real-world Java Spring Boot production environment:

1. The External Load Balancer Level (In Front)

- **What it does:** A highly lightweight, blazingly fast hardware or cloud-routing load balancer (like AWS Application Load Balancer) sits at the edge of your network. It holds your public domain's SSL certificate (SSL Termination) and acts as the entry point for api.yourdomain.com.
- **How it routes:** It doesn't look at complex application logic or user databases. It simply sees incoming TCP/HTTP traffic and passes it using an algorithm (like Round Robin) to **Gateway Instance #1** or **Gateway Instance #2**.

2. The API Gateway Level (The Brains)

- **What it does:** Now that a specific gateway instance (e.g., Spring Cloud Gateway) has received the request safely, it performs the heavy cryptographic and computational tasks:
    - Decrypts/Validates the user's **JWT Token** (Authentication).
    - Checks if the user is an ADMIN or USER (Authorization).
    - Applies **Rate Limiting** (via Redis) to make sure the user isn't abusing the API.
- **The "Next Step" Routing:** Once the gateway approves the request, it looks at the path (e.g., /orders/checkout). It asks a Service Registry (like **Netflix Eureka** or Kubernetes DNS) where the healthy instances of the Order-Service are.

3. The Internal Load Balancer Level (Behind the Gateway)

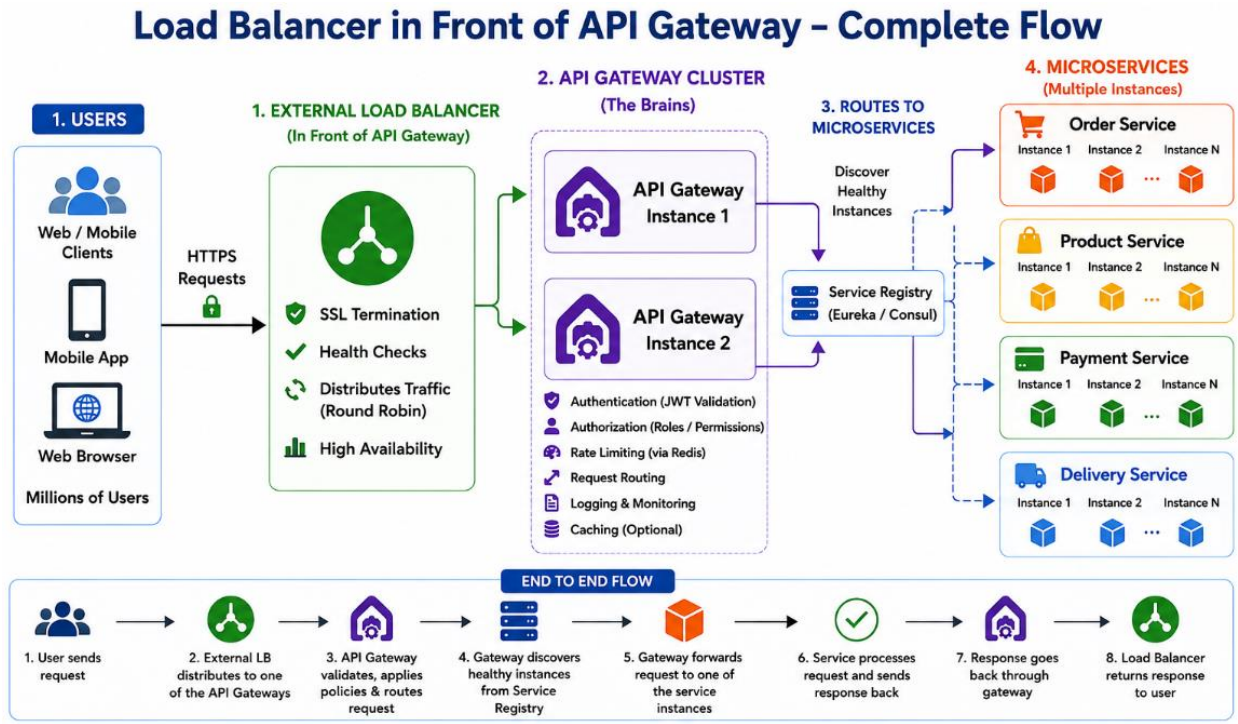
This is likely what you were thinking of! **You are not wrong; a form of load balancing *does* happen after the gateway.**

- Once the API Gateway knows the request is valid and needs to go to the Order-Service, it acts as a **software load balancer** (using libraries like **Spring Cloud LoadBalancer**).
- It chooses one of the many running instances of the Order-Service and forwards the request internally.

LB vs. Internal Routing

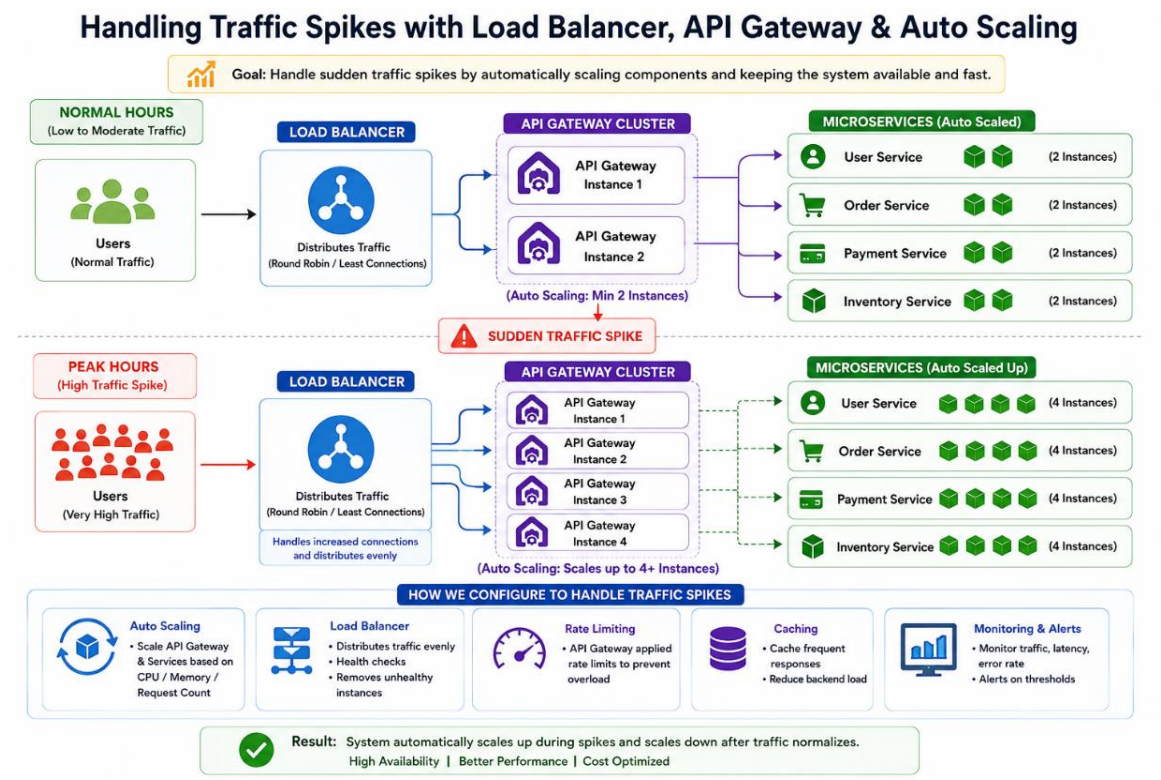
Criteria	External Load Balancer (In Front of Gateway)	Internal Load Balancing (After Gateway)
Primary Job	Protects the API Gateway from crashing by spreading raw traffic across multiple gateway instances.	Routes authorized, clean requests to the correct backend microservice instances.
Tech Example	AWS ALB, Cloudflare, NGINX, F5 Big-IP.	Spring Cloud LoadBalancer, Core Kubernetes Kube-Proxy.
Awareness	Knows only about network health and Gateway instances.	Knows about API paths, user claims, and individual microservice instances.

The Takeaway for your Interview: Saying *"I will put a Load Balancer in front of the API Gateway cluster to prevent a single point of failure, and let the API Gateway handle internal service-level load balancing"* is the ultimate, highest-scoring architectural answer you can give.



2. Traffic Spikes During Certain Hours

Question : Your application experiences spikes during certain times of the day. How would you configure the load balancer and API gateway?

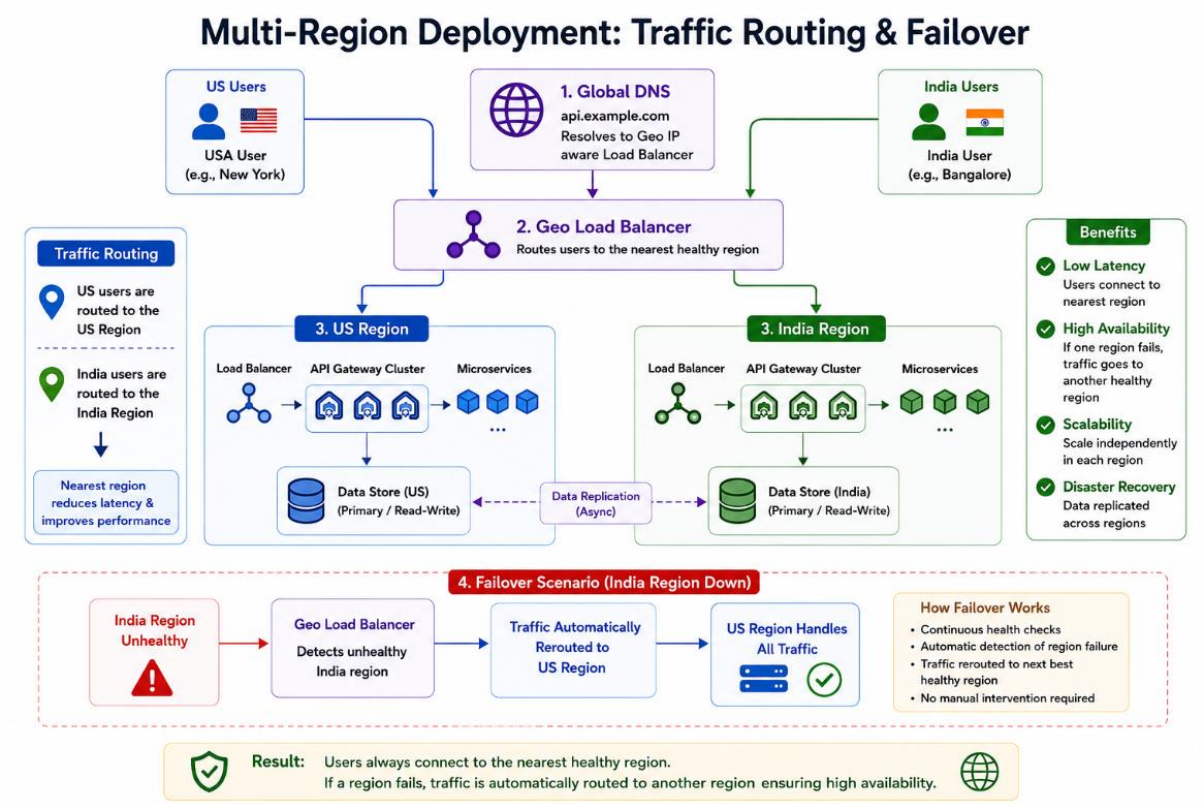


Answer : I would use: **Auto Scaling.**

3. Multi-Region Deployment

Question : You have services running in multiple regions worldwide. How would you route traffic and handle failover?

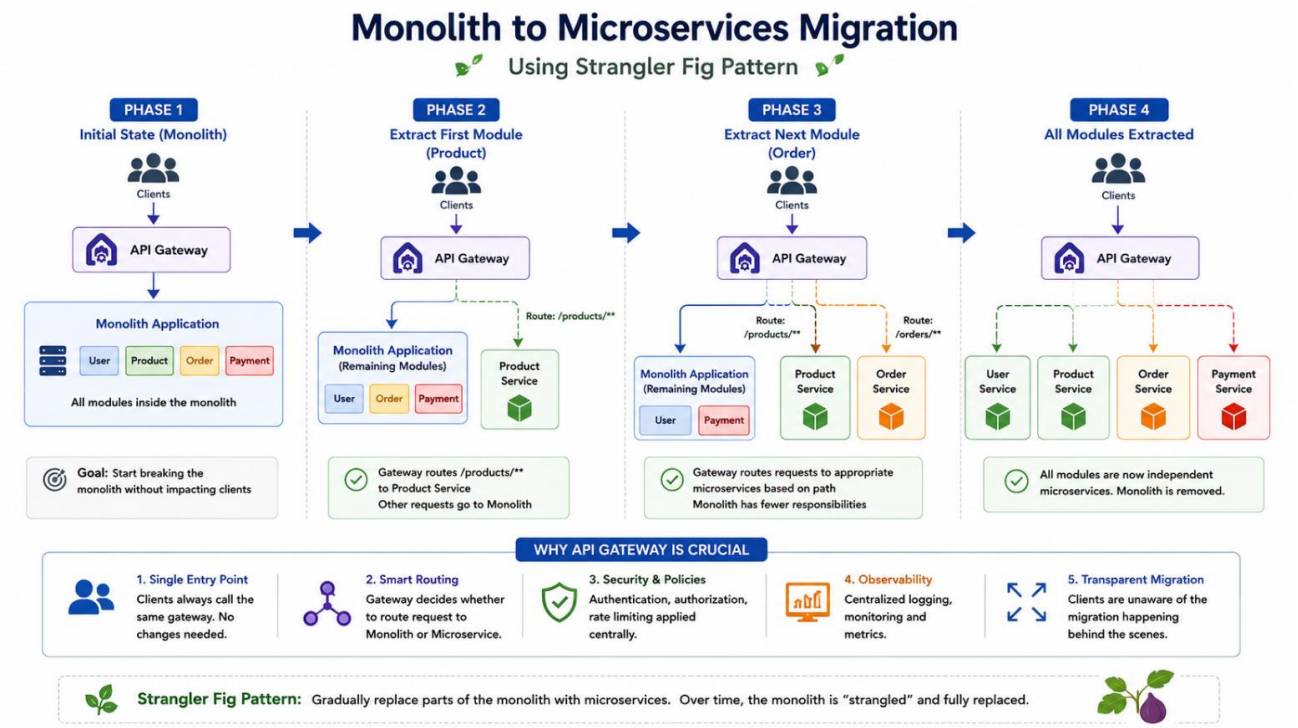
Answer : I would use Geo Load Balancer: Routes user requests based on the users region to nearest healthy region.



4. Monolith → Microservices Migration

Question : Your organization is moving from a monolith to microservices.

Answer : I would use the **Strangler Fig Pattern**.



Strangler pattern : The Strangler pattern is like slowly replacing an old system with a new one. Instead of breaking the old system all at once, we move one part at a time into microservices.

Example:
Imagine we have a big monolithic food delivery app. It has login, restaurant listing, orders, payments, and delivery tracking in one single block of code.

Instead of rewriting everything, we start with one part, say Payments.

1. We build a new Payment microservice.
2. We redirect only the payment requests from the old system to the new Payment service.
3. The rest of the features keep using the old system.
4. Over time, we move other parts like Orders and Delivery the same way.

This pattern lets us modernize the system step by step without shutting it down. It reduces risk and makes the transition smooth.

It's called "Strangler" because the new microservices slowly **replace and surround** the old system until the old system is no longer needed.

What are benefits of using Strangler Pattern?

The Strangler Pattern gives a safe way to move from an old system to a new system step by step. Instead of replacing everything at once, we slowly build the new system around the old one and move one feature at a time.

Over time, the old system gets smaller and eventually disappears.

Benefits of the Strangler Pattern

1. No big shutdown : we do not need to stop the existing system, The business can continue normally while we build the new system piece by piece. This avoids long downtime and angry users.

2. Lower risk : Replacing a full system at once is risky. If something goes wrong, the whole system fails. With the Strangler Pattern, we move one feature at a time. If a new feature fails, only that part is affected, not the entire system. This lowers the risk of failure.

3. Works well with modernisation : It allows us to slowly modernise.

For example, we can move one module from a monolithic Java application to microservices. Or we can move from REST to GraphQL, or from a SQL database to NoSQL, one step at a time.

4. Easy rollback : If something goes wrong in the new version of a feature, we can simply route traffic back to the old feature. Rollback becomes easy and safe.

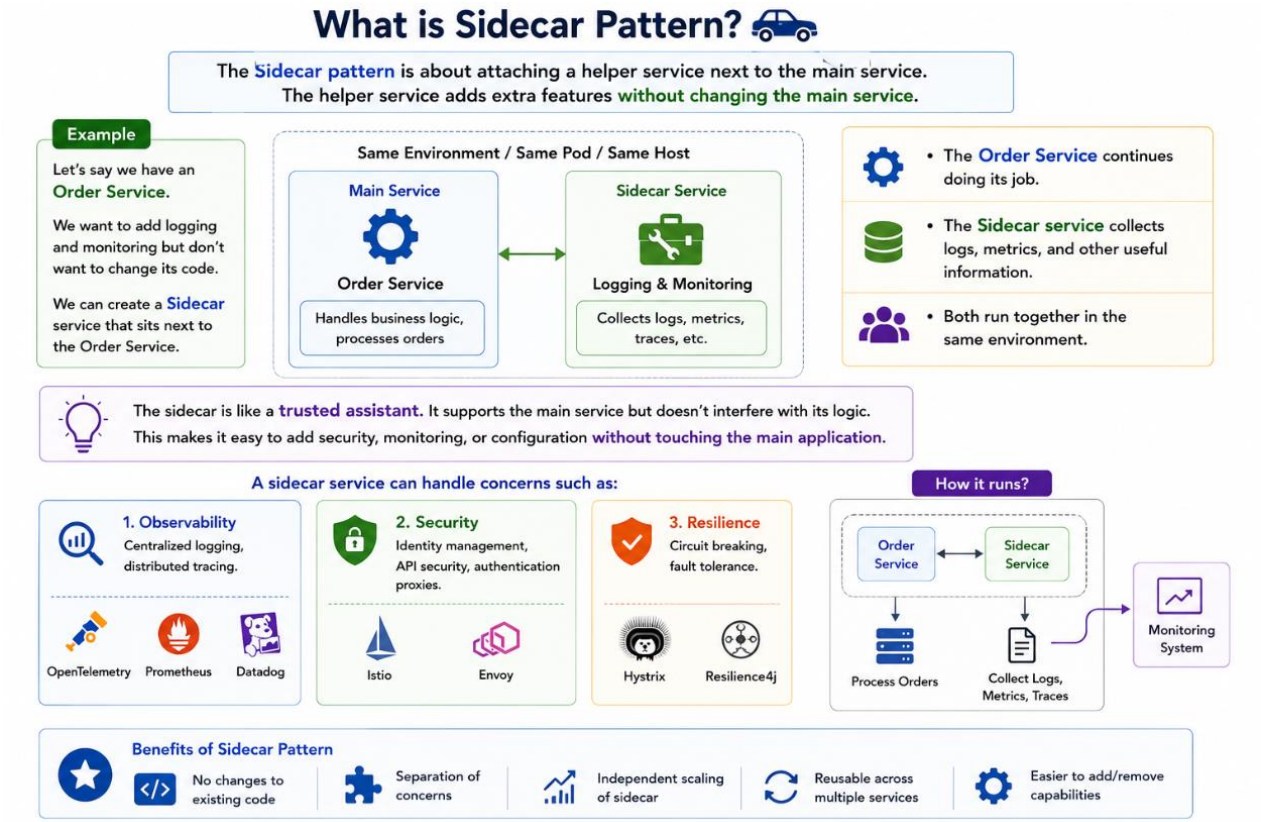
5. Keeps business value running

The team can deliver improvements continuously instead of waiting for a big project release after 6–12 months. This keeps users happy and helps the business see value early.

What are disadvantages or challenges of Strangler pattern?

1. It takes time. Since we move feature by feature, full migration can take months or even years.
2. Extra complexity. During migration, we maintain both old and new systems, which increases effort.
3. Data syncing issues. Old and new services may use different databases, so keeping data in sync becomes difficult.

What is SideCar Pattern ?



You use the **Strangler Fig Pattern** when you want to migrate a system by gradually replacing old monolithic features with new microservices until nothing is left of the monolith.

The **Sidecar Pattern**, on the other hand, is not used for migration. Instead, it is used **to add common helper features to a microservice without touching or changing its actual application code.**

Think of it like a motorcycle sidecar: the motorcycle (your core business logic) focuses entirely on driving forward, while the sidecar (the helper service) sits right next to it, sharing the same space, and carrying the extra gear.

Why and When to Use the Sidecar Pattern

In a microservices architecture, every service needs basic, repetitive infrastructure capabilities like logging, security, and monitoring. If you code these directly into every single microservice, you end up duplicating code and running into issues if different services are written in different languages (e.g., Java, Go, Node.js).

The Sidecar pattern solves this by isolating these tasks into a separate process that runs right alongside your main application inside the same host or container pod.

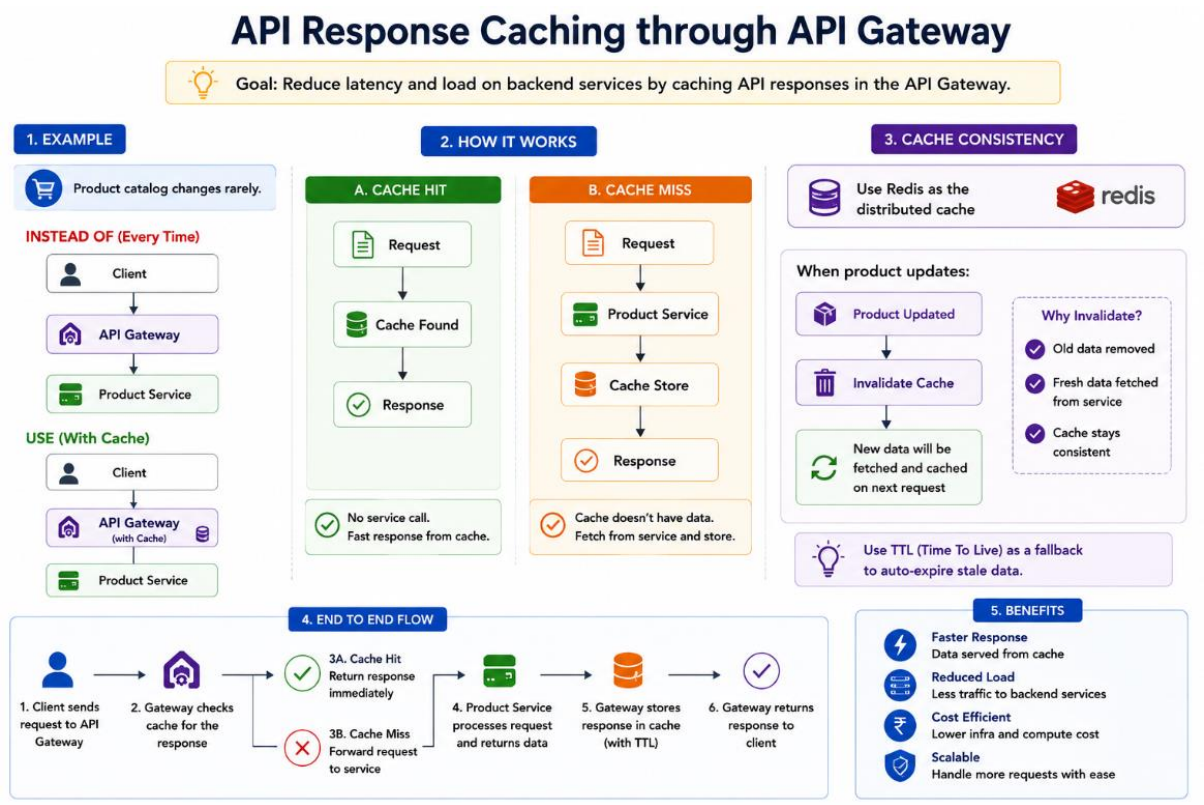
Pattern	Primary Purpose	Real-world Analogy
Strangler Fig	**Migration & Evolution:** Used to safely dismantle a monolith and replace it with microservices over time.	Building a new modern highway right next to an old bumpy road, slowly redirecting traffic until the old road is closed.
Sidecar	**Modularization & Operational Concerns:** Used to offload non-business logic (networking, logs, security) away from the main app.	A personal assistant who travels everywhere with an executive, handling their scheduling and security so the executive can focus purely on business decisions.

5. API Response Caching through API Gateway

Question : How would you implement caching through API Gateway?

Answer : In a microservices architecture, some data is requested frequently but changes very rarely. A common example is a product catalog in an e-commerce application. Fetching the same data from backend services for every request increases latency and puts unnecessary load on services.

To solve this, the API Gateway can act as a caching layer.



Real-World Example: Product Catalog

Imagine an e-commerce application where thousands of users continuously browse products.

Without caching, every request follows this flow:

The Product Service must process the same request repeatedly, even though the product data hasn't changed.

With API Gateway caching enabled:

The gateway first checks whether the requested data is already available in cache.

Cache Hit Scenario

A Cache Hit occurs when the requested data already exists in the cache.

In this case:

1. No call is made to the Product Service.
2. Response is returned immediately.
3. Latency is significantly reduced.
4. Backend resources are saved.

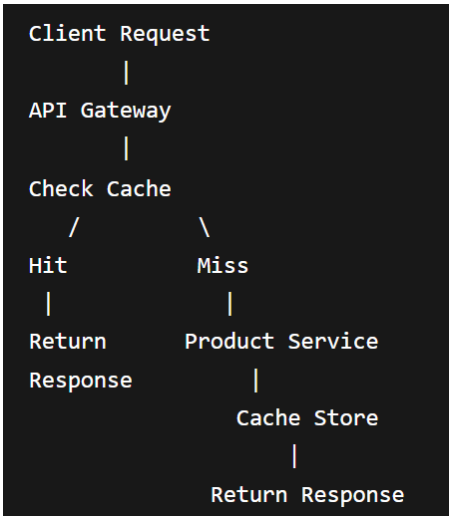
This is the fastest possible path.

Cache Miss Scenario

A Cache Miss occurs when the requested data is not available in cache.

In this case:

1. The API Gateway forwards the request to the Product Service.
2. The Product Service processes the request and returns data.
3. The API Gateway stores the response in cache.
4. The response is returned to the client.
5. Future requests can be served directly from cache.



Interview Answer

"I would implement caching at the API Gateway layer using Redis. When a request arrives, the gateway first checks the cache. If the data exists (cache hit), it immediately returns the response without calling the Product Service. If the data is not present (cache miss), the gateway fetches the data from the Product Service, stores it in Redis, and returns the response. To maintain consistency, cache entries are invalidated whenever product data changes, and TTL can be used to automatically expire stale data."

REST API Flow – Weather API